Sri Devaraj Urs Educational Trust (R.)

R. L. JALAPPA INSTITUTE OF TECHNOLOGY

(Approved by AICTE, New Delhi, Affiliated to VTU, Belagavi & Accredited by NAAC "A" Grade)

Kodigehalli, Doddaballapur- 561 203

Department of CS&E - PG

Subject Code: MCS103
Subject Name: DATA STRUCTURES AND APPLICATIONS
Module Number: 01
Name of the Module: SEARCH TREES **Scheme:** 2024

Prepared by: Dr. Mamatha C M

Professor

Institute Vision

To be a premier Institution by imparting quality Technical education, Professional Training andResearch.

Institute Mission

M1: To provide an outstanding Teaching, Learning and Research environment through InnovativePractices in Quality Education.

M2: Develop Leaders with high level of Professionalism to have career in the Industry, Zeal forHigher Education, focus on Entrepreneurial and Societal activities.

Department Vision- PG

To nurture students with advanced expertise and research-oriented skills in Computer Science and Engineering, empowering them to drive technological innovation and thrive in an evolving global landscape.

Department Mission- PG

M1: To foster advanced skills in specialized domains of Computer Science and Engineering, equippingstudents with the necessary expertise to address contemporary challenges and meet the evolving demands of the global industry.

M2: To promote cutting-edge research and technological innovation, while cultivating entrepreneurship and consultancy skills that empower students to contribute to the technological needs of industries, governments, and society.

PROGRAMME SPECIFIC OUTCOMES (PSOs)

PSO1: Students will have a knowledge of Advanced Software, Hardware, Network Models, Algorithms

PSO2: Students will be able to develop applications in the areas related to Artificial Intelligence, Machine Learning, Data Science and IoT for efficient design of computer-based systems.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Our Graduates will have prospective careers in the IT Industry.

PEO2: Our Graduates will exhibit a high level of Professionalism and Leadership skills in work Environment.

PEO3: Our Graduates will pursue Research, and focus on Entrepreneurship.

Module-1

Search Trees: Two Models of Search Trees. General Properties and Transformations. Height of a Search Tree. Basic Find, Insert, and Delete. Returning from Leaf to Root. Dealing with Non unique Keys. Queries for the Keys in an Interval. Building Optimal Search Trees. Converting Trees into Lists. Removing a Tree. Balanced Search Trees: Height-Balanced Trees. Weight-Balanced Trees. (a, b)- And B-Trees. Red-Black Trees and Trees of Almost Optimal Height. Top-Down Re balancing for Red-Black Trees.

Two Models of Search Trees

If we compare in each node the query key with the key contained in the node and follow the left branch if the query key is smaller and the right branch if the query key is larger, then what happens if they are equal? The two models of search trees are as follows:

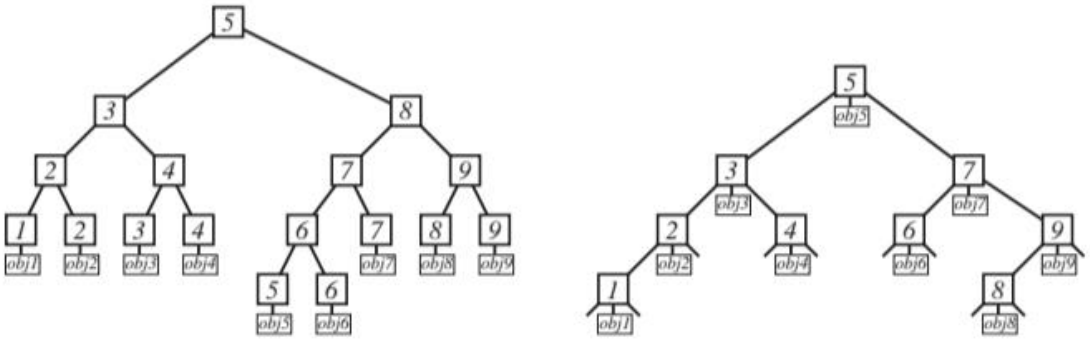
1. Take left branch if query key is smaller than node key; otherwise take the right branch, until you reach a leaf of the tree. The keys in the interior node of the tree are only for comparison; all the objects are in the leaves.
2. Take left branch if query key is smaller than node key; take the right branch if the query key is larger than the node key; and take the object contained in the node if they are equal.

Difference between Model 1 and Model 2

- In model 1, the underlying tree is a binary tree, whereas in model 2, each tree node is really a ternary node with a special middle neighbor.
- In model 1, each interior node has a left and a right subtree (each possibly a leaf node of the tree), whereas in model 2, we have to allow incomplete nodes, where left or right subtree might be missing, and only the comparison object and key are guaranteed to exist.
- In model 1, traversing an interior node requires only one comparison, whereas in model 2, we need two comparisons to check the three possibilities.

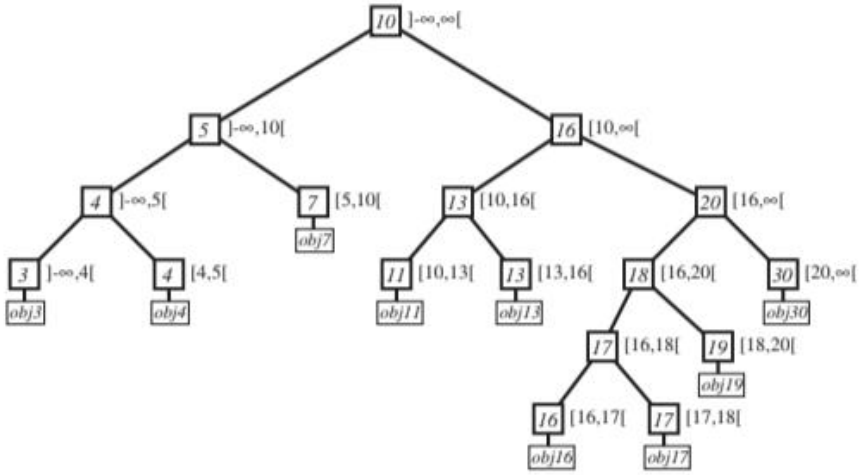
A tree of **model 1** consists of nodes of the following structure:

```
typedef struct tr_n_t {key_t key;
struct tr_n_t *left;
struct tr_n_t *right;
/* possibly additional information */
} tree_node_t;
```



SEARCH TREES OF MODEL 1 AND MODEL 2

General Properties and Transformations



INTERVALS ASSOCIATED WITH NODES IN A SEARCH TREE

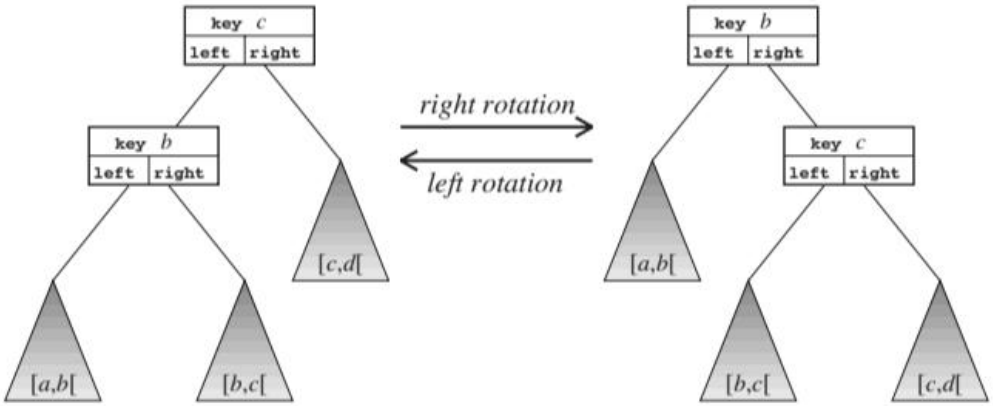
In a correct search tree, we can associate each tree node with an interval, the interval of possible key values that can

be reached through this node. The interval of root is]–∞,∞[, and if *n is an interior node associated with interval [a, b[, then n->key ∈ [a, b[, and n->left and n->right have as associated intervals [a, n->key[and [n->key, b[. With the exception of the intervals starting in −∞, all these intervals are half-open, containing the left endpoint but not the right endpoint. This implicit structure on the tree nodes is very helpful in understanding the operations on the trees.
There are two operations – the left and right rotations – that transform a correct search tree in a different correct search tree for the same set. They are used as building blocks of more complex tree transformations because they are easy to implement and universal.
The following code does a **left rotation** around *n:

```
void left_rotation(tree_node_t *n)
{ tree_node_t *tmp_node;
key_t tmp_key;
tmp_node = n->left;
tmp_key = n->key;
n->left = n->right;
n->key = n->right->key;
n->right = n->left->right;
n->left->right = n->left->left;
n->left->left = tmp_node;
n->left->key = tmp_key;
}
```

The **right rotation** is exactly the inverse operation of the left rotation.

```
void right_rotation(tree_node_t *n)
{ tree_node_t *tmp_node;
key_t tmp_key;
tmp_node = n->right;
tmp_key = n->key;
n->right = n->left;
n->key = n->left->key;
n->left = n->right->left;
n->right->left = n->right->right;
n->right->right = tmp_node;
n->right->key = tmp_key;
}
```



LEFT AND RIGHT ROTATIONS

Height of a Search Tree

The height of a search tree is the maximum length of a path from the root to a leaf – the maximum taken over all leaves. the maximum number of leaves of a search tree of height h is 2h. And at the other end, the minimum number of leaves is h + 1because a tree of height h must have at least one interior node at each depth 0,...,h − 1, and a tree with h interior nodes has h + 1 leaves. Together, this gives the bounds.

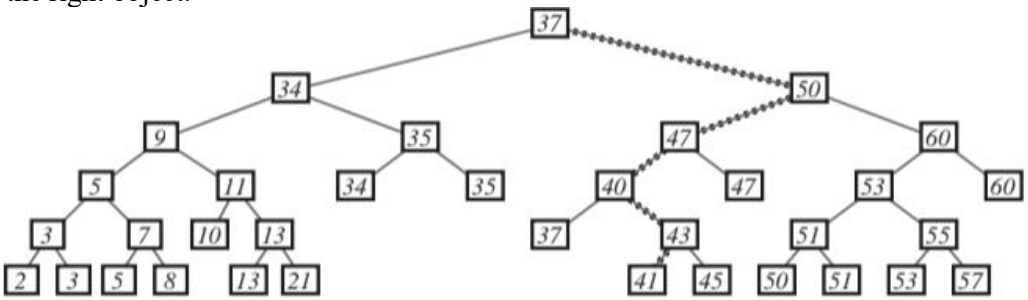
Basic Find, Insert, and Delete

The search tree represents a set of (key, object) pairs, so it must allow some operations with this set. The most important operations that any search tree needs to support are as follows:

{ find(tree, query key): Returns the object associated with query key, if there is one;
{ insert(tree, key, object): Inserts the (key, object) pair in the tree; and
{ delete(tree, key): Deletes the object associated with key from the tree.

Find

The simplest operation is the find: one just follows the associated interval structure to the leaf, which is the only place that could hold the right object.



SEARCH TREE AND SEARCH PATH FOR UNSUCCESSFUL `find(tree, 42)`

Code:

```
object_t *find(tree_node_t *tree,
key_t query_key)
{ if( tree->left == NULL ||
(tree->right == NULL &&
tree->key != query_key ) )
return(NULL);
else if (tree->right == NULL &&
tree->key == query_key )
return( (object_t *) tree->left );
else
{ if( query_key < tree->key )
return( find(tree->left, query_key) );
else
return( find(tree->right, query_key) );
}
}
```

Insert

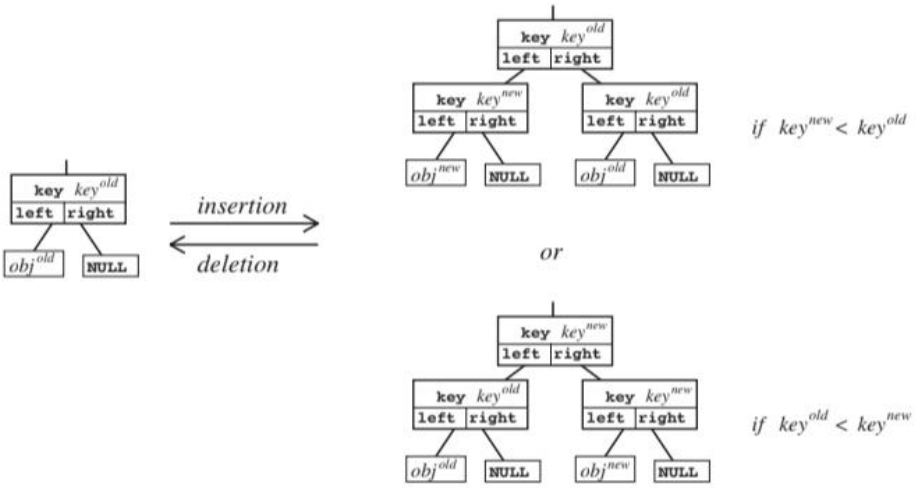
The insert operation starts out the same as the find, but after it finds the correct place to insert the new object, it has to create a new interior node and a new leaf node and put them in the tree. We assume, as always, that there are functions get node and return node available assuming all keys are unique

```
int insert(tree_node_t *tree, key_t new_key,

object_t *new_object)
{ tree_node_t *tmp_node;
if( tree->left == NULL )
{ tree->left = (tree_node_t *) new_object;
tree->key = new_key;
tree->right = NULL;
}
else
{ tmp_node = tree;
while( tmp_node->right != NULL )
{ if( new_key < tmp_node->key )
tmp_node = tmp_node->left;
else
tmp_node = tmp_node->right;

}
/* found the candidate leaf. Test whether
key distinct */
if( tmp_node->key == new_key )
return( -1 );
/* key is distinct, now perform the insert */
{ tree_node_t *old_leaf, *new_leaf;
old_leaf = get_node();
old_leaf->left = tmp_node->left;
old_leaf->key = tmp_node->key;
old_leaf->right = NULL;
```

```
new_leaf = get_node();
new_leaf->left = (tree_node_t *)
new_object;
new_leaf->key = new_key;
new_leaf->right = NULL;
if( tmp_node->key < new_key )
{ tmp_node->left = old_leaf;
tmp_node->right = new_leaf;
tmp_node->key = new_key;
}
else
{ tmp_node->left = new_leaf;
tmp_node->right = old_leaf;
}
}
}
return( 0 );
}
```



INSERTION AND DELETION OF A LEAF

The delete operation is even more complicated because when we are deleting a leaf, we must also delete an interior node above the leaf. For this, we need to keep track of the current node and its upper neighbor while going down in the tree. Also, this operation can lead to an error if there is no object with the given key.

```
object_t *delete(tree_node_t *tree,
key_t delete_key)
{ tree_node_t *tmp_node, *upper_node,
*other_node;
object_t *deleted_object;
if( tree->left == NULL )
return( NULL );
else if( tree->right == NULL )
{ if( tree->key == delete_key )
{ deleted_object =

(object_t *) tree->left;

tree->left = NULL;
return( deleted_object );
}
else
return( NULL );
}
else
{ tmp_node = tree;
while( tmp_node->right != NULL )
{ upper_node = tmp_node;
if( delete_key < tmp_node->key )
```

```
{ tmp_node = upper_node->left;
other_node = upper_node->right;
}
else
{ tmp_node = upper_node->right;
other_node = upper_node->left;
}
}
if( tmp_node->key != delete_key )
return( NULL );
else
{ upper_node->key = other_node->key;
upper_node->left = other_node->left;
upper_node->right = other_node->right;
deleted_object = (object_t *)
tmp_node->left;
return_node( tmp_node );
return_node( other_node );
return( deleted_object );
}
```

Returning from Leaf to Root

To perform some update or rebalancing operations on the nodes of this path. And these operations need to be done in that order, with the leaf first and the root last. But without additional measures, the basic search-tree structure we described does not contain any way to reconstruct this sequence. There are several possibilities to save this information.

1. **A stack:** If we push pointers to all traversed nodes on a stack during descent to the leaf, then we can take the nodes from the stack in the correct (reversed) sequence afterward. It does not put any additional information into the tree structure. Also, the maximum size of the stack needed is the height of the tree, and so for the balanced search trees, it is logarithmic in the size of the search tree **O(log n)**. An array-based stack for 200 items is really enough for all realistic applications because we will never have two 100 items. This is also the solution implicitly used in any recursive implementation of the search trees.
2. **Back pointers:** If each node contains not only the pointers to the left and right subtrees, but also **a pointer to the node above it**, then we have a path up from any node back to the root. This requires **an additional field in each node**. This pointer also has to be corrected in each operation, which makes it again a source of possible programming errors.
3. **Back pointer with lazy update:** If we have in each node an entry for the pointer to the node above it, but we actually **enter the correct value only when the tree reaches leaf node**.

Dealing with Nonunique Keys

In database applications, we might have to store many objects with the same key value; there it is a quite unrealistic assumption that each object is uniquely identified by each of its attribute values, but there are queries to list all objects with a given attribute value. So any realistic search tree has to deal with this situation. The correct reaction is as follows:

{ **find** returns all objects whose key is the given query key in output-sensitive time **O(h + k)**, where **h is the height** of the tree and **k is the number of elements that find returns**.

{ **insert** always inserts correctly in time **O(h)**, where h is the height of the tree.

{ **delete** deletes all items of that key in time **O(h)**, where h is the height of the tree.

Find just produces all elements of that list; insert always inserts at the beginning of the list; only delete in time independent of the number of deleted items requires additional information. For this, **we need an additional node between the leaf and the linked list,** which contains pointers to the beginning and to the end of the list; then we can transfer the entire list with **O(1)** operations to the free list of our dynamic memory allocation structure.

Queries for the Keys in an Interval

We given a key interval [a, b[and want to find all keys that are contained in this interval. If the keys are subject to small errors, we might not know the key exactly, so we want the nearest key value or the next larger or next smaller key. Without such an extension, our find operation just answers that there is no object with the given key in the current set.

1. We can organize the leaves into a doubly linked list and then we can move in O(1) time from a leaf to the next larger and the next smaller leaf. This requires a change in the insertion and deletion functions to maintain the list. The query method is also almost the same; it takes O(k) additional time if it lists a total of k keys in the interval.
2. We go down the tree with the query interval instead of the query key. Then we go left if [a, b[< node->key; right if node->key ≤ [a, b[; and sometimes we have to go both left and right if a < node->key ≤ b. We store all those branches

that we still need to explore on a stack. The nodes we visit this way are the nodes on the search path for the beginning of the query interval a, the search path for its end b, and all nodes that are in the tree between these paths. If there are i interior nodes between these paths, there must be at least i + 1 leaves between these paths. So if this method lists k leaves, the total number of nodes visited is at most twice the number of nodes visited in a normal find operation plus O(k). Thus, this method is slightly slower than the first method but requires no change in the insert and delete operations.

Building Optimal Search Trees

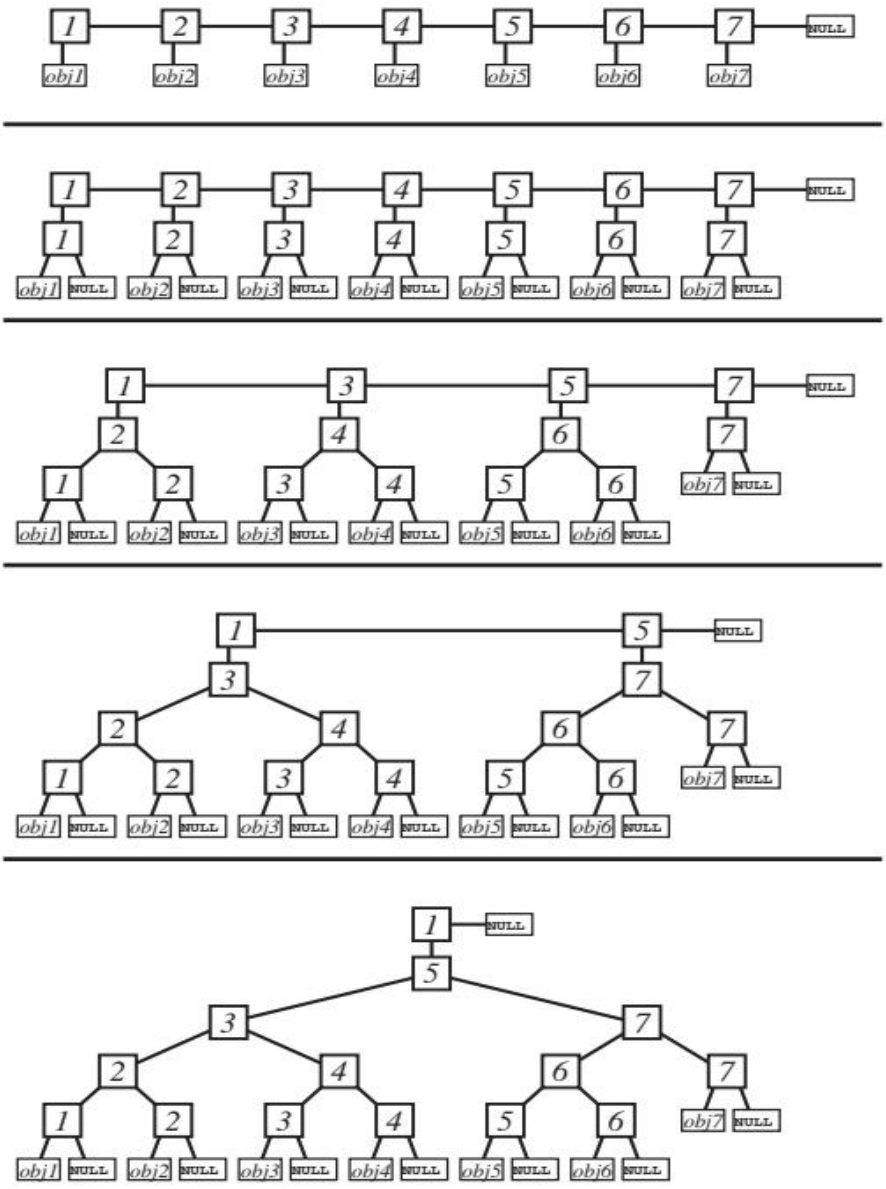
The primary criterion is the height; because a search tree of height h has at most 2h leaves, an optimal search tree for a set of n items has height log n, where the log, as always, is taken to base 2.

There are two natural ways to construct a search tree of optimal height from a sorted list: bottom-up and top-down.

The bottom-up construction is easier: one views the initial list as list of one-element trees. Then one goes repeatedly through the list, joining two consecutive trees, until there is only one tree left. This requires only a bit of bookkeeping to insert the correct comparison key in each interior node. The disadvantage of this method is that the resulting tree, although of optimal height, might be quite unbalanced: if we start with a set of n = 2m + 1 items, then the root of the tree has on one side a subtree of 2m items and on the other side a subtree of 1 item.

Next is the code for the bottom-up construction. We assume here that the list items are themselves of type tree node t, with the left entry pointing to the object, the key containing the object key, and the right entry pointing to the next item, or NULL at the end of the list. We first create a list, where all the nodes of the previous list are attached as leaves, and then maintain a list of trees, where the key value in the list is the smallest key value in the tree below it.

The first half of the algorithm, duplicating the list and converting all the original list nodes to leaves, takes obviously **O(n);** it is just one loop over the length of the list. The second half has a more complicated structure, but in each execution of the body of the innermost loop, one of the **n interior nodes** created in the first half is removed from the current list and put into a finished subtree, so the innermost part of the two nested loops is **executed only n times**

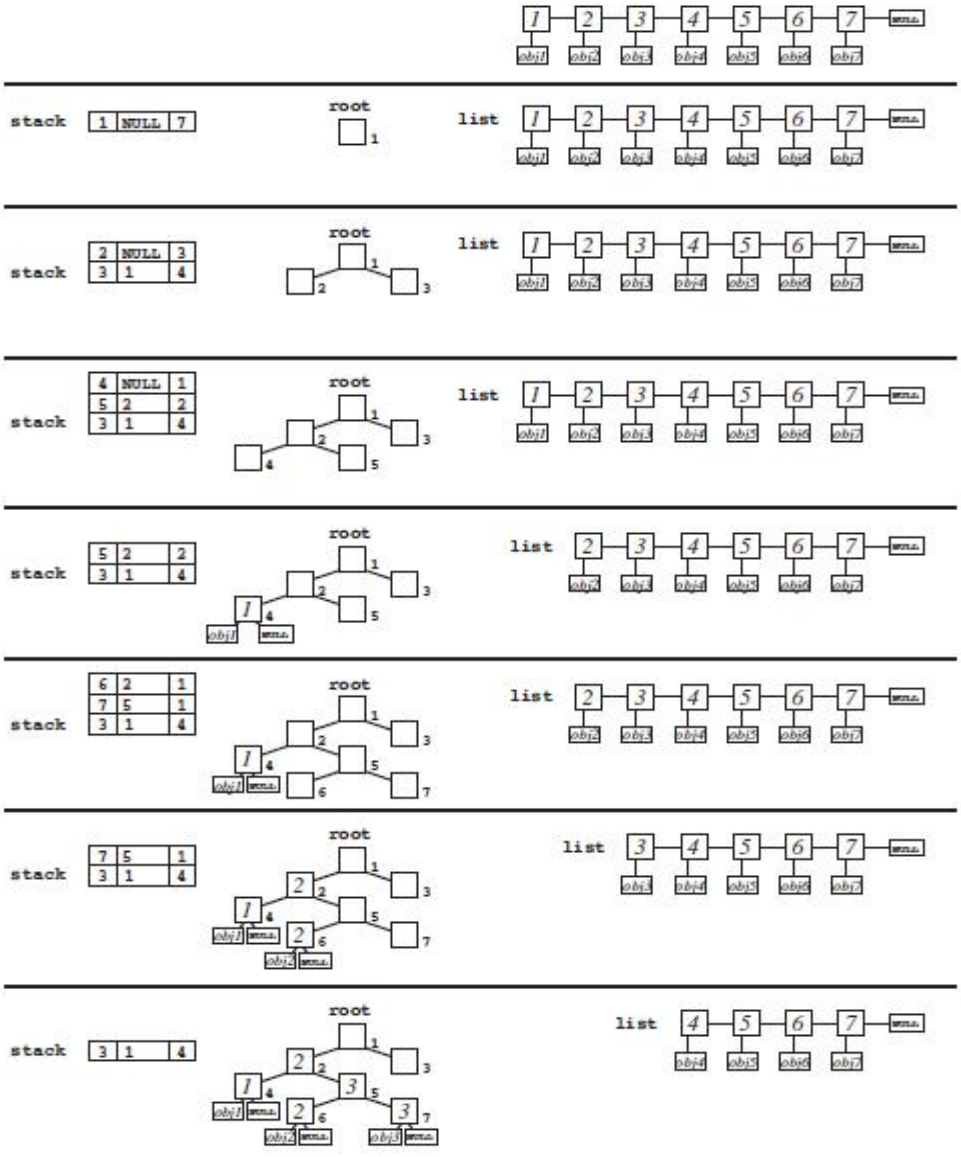


Bottom-Up Construction of an Optimal Tree from a Sorted List

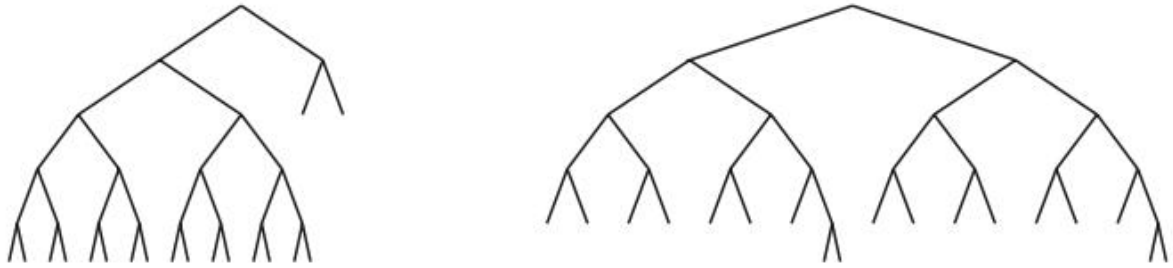
The **top-down** construction is easiest to describe recursively: divide the data set in the middle, create optimal trees for

the lower and the upper halves, and join them together. This division is very balanced; in each subtree the number of items left and right differs by at most one, and it also results in a tree of optimal height. But if we implement it this way, and the data is given as list, it takes (n log n) time, because we get an overhead of (n) in each recursion step to find the middle of the list. But there is a nice implementation with O(n) complexity using a stack. We write here the generic stack operations push, pop, stack empty, create stack, remove stack to illustrate how this method works. In a concrete implementation they should be replaced by one of the methods discussed in Chapter 1. In this case an array-based stack is the best method, and one should declare the stack as local array in the function, avoiding all function calls.

The idea of our top-down construction is that we first construct the tree "in the abstract," without filling in any key values or pointers to objects. Then we do not need the time to find the middle of the list; we just need to keep track of the number of elements that should go into the left and right subtrees. We can build this abstract tree of the required shape easily using a stack. We initially put the root on the stack, labeled with the required tree size; then we continue, until the stack is empty, to take nodes from the stack, attach them to two newly created nodes labeled with half the size, and put the new nodes again on the stack. If the size reaches one, we have a leaf, so node should not be put back on the stack but should be filled with the next key and object from the list. The problem is to fill in the keys of the interior nodes, which become available only when the leaf is reached. For this, each item on the stack needs two pointers, one to the node that still needs to be expanded and one to the node higher up in the tree, where the smallest key of leaves below that node should be inserted as comparison key. Also, each stack item contains a number, the number of leaves that should be created below that node. When we perform that step of taking a node from the stack and creating its two lower neighbors, the right-lower neighbor should and always go first on the stack, and then the left, so that when we reach a leaf, it is the leftmost unfinished leaf of the tree. This pointer for the missing key value propagates into the left subtree of the current node (where that smallest node comes from), whereas the smallest key from the right subtree should become the comparison key of the current node.For this stack, an array-based stack of size 100 will be entirely sufficient because the size of the stack is the height of the tree, which is log n, and we can assume $n < 2^{100}$.



Top-Down Construction of an Optimal Tree from a Sorted List First Steps, until the Left Half Is Finished



BOTTOM-UP AND TOP-DOWN OPTIMAL TREE WITH 18 LEAVES

Converting Trees into Lists

Using a stack for a trivial depth-first search enumeration of the leaves in decreasing order, which we insert in front of the list. This converts in $O(n)$ time a search tree with n leaves into a list of n elements in increasing order. Again we write the generic stack functions, which in the specific implementation should be replaced by the correct method. If one knows in advance that the height of the tree is not too large, an array is the preferred method; the size of the array needs to be at least as large as the height of the tree.

```
tree_node_t *make_list(tree_node_t *tree)
{ tree_node_t *list, *node;
if( tree->left == NULL )
{ return_node( tree );
return( NULL );
}
else
{ create_stack();
push( tree );
list = NULL;
while( !stack_empty() )
{ node = pop();
if( node->right == NULL )
{ node->right = list;
list = node;
}
else
{ push( node->left );
push( node->right );
return_node( node );
}
}
return( list );
}
}
```

Removing a Tree

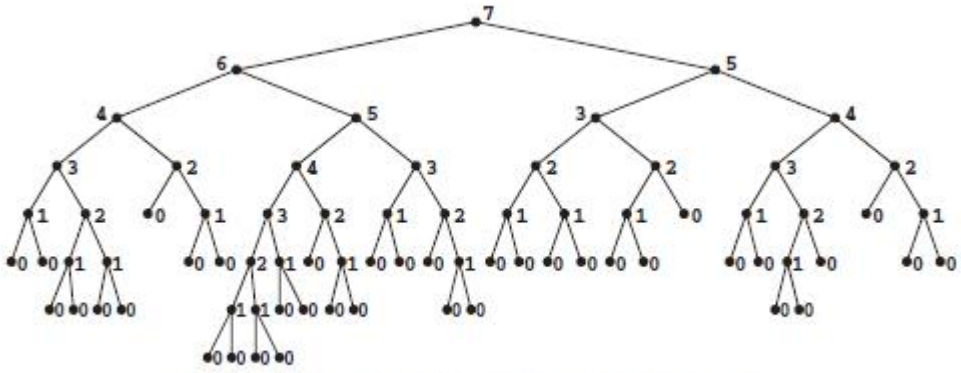
We also need to provide a method to remove the tree when we no longer need it. As we already remarked for the stacks, it is important to free all nodes in such a dynamically allocated structure correctly, so that we avoid a memory leak. We cannot expect to remove a structure of potentially large size in constant time, but time linear in the size of the structure, that is, constant time per returned node, is easily reached. An obvious way to do this is using a stack, analogous to the previous method to covert a tree into a sorted list. A more elegant method is the following:

```
void remove_tree(tree_node_t *tree)
{ tree_node_t *current_node, *tmp;
if( tree->left == NULL )
return_node( tree );
else
{ current_node = tree;
while(current_node->right != NULL )
{ if( current_node->left->right == NULL )
{ return_node( current_node->left );
tmp = current_node->right;
return_node( current_node );
current_node = tmp;
}
```

```
else
{ tmp = current_node->left;
current_node->left = tmp->right;
tmp->right = current_node;
current_node = tmp;
}
}
return_node( current_node );
}
}
```

Height-Balanced Trees[AVL- trees]

A tree is height-balanced if, in each interior node, the height of the right subtree and the height of the left subtree differ by at most 1. The height information must be corrected whenever the tree changes and must be used to keep the tree height balanced.



HEIGHT-BALANCED TREE WITH NODE HEIGHTS

The tree changes only by insert and delete operations, and by any such operation, the height can change only for the nodes that contain the changed leaf in their subtree, that is, only for the nodes on the path from the root to the changed leaf. At the leaf that was changed, or in the case of an insert, the two neighboring leaves, the height is 0. Now following the path up to the root, we have in each node the following situation: the height information in the left and right subtrees is already correct, and both subtrees are already height balanced: one because we restored balance in the previous step of going up and the other because nothing changed in that subtree. Also, the heights of both subtrees differ by at most 2 because previous to the update operation, the height differed by at most 1 and the update changed the height by at most 1. We now have to balance this node and update its height before we can go further up.

If *n is the current node, there are the following possibilities:

1. |n->left->height − n->right->height| ≤ 1.
In this case, no rebalancing is necessary in this node. If the height also did not change, then from this node upward nothing changed and we can finish rebalancing; otherwise, we need to correct the height of *n and go up to the next node.

2. |n->left->height − n->right->height| = 2.
In this case, we need to rebalance *n. This is done using the rotations introduced in Section 2.2. The complete rules are as follows:

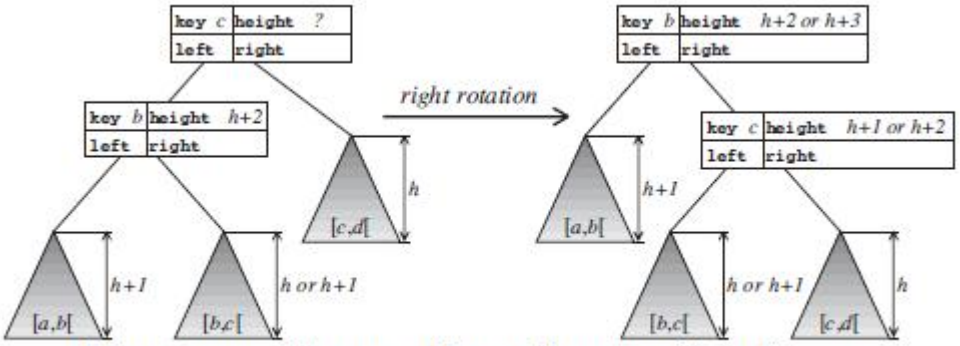
2.1 If n->left->height = n->right->height + 2 and
n->left->left->height = n->right->height + 1.
Perform right rotation around n, followed by recomputing the height in n->right and n.

2.2 If n->left->height = n->right->height + 2 and
n->left->left->height = n->right->height.
Perform left rotation around n->left, followed by a right rotation around n, followed by recomputing the height in n->right, n->left, and n.

2.3 If n->right->height = n->left->height + 2 and
n->right->right->height = n->left->height + 1.
Perform left rotation around n, followed by recomputing the height in n->left and n.

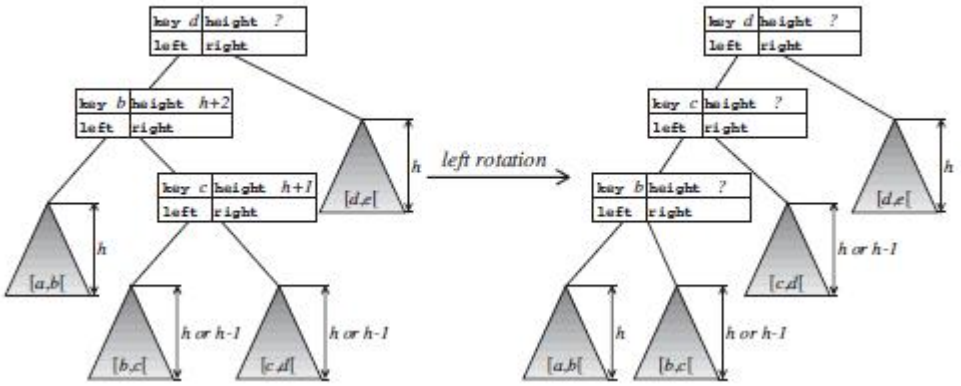
2.4 If n->right->height = n->left->height + 2 and
n->right->right->height = n->left->height.
Perform right rotation around n->right, followed by a left rotation

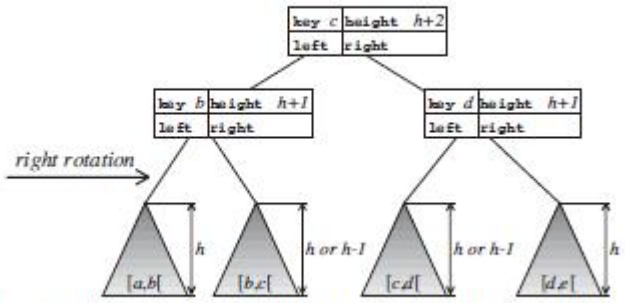
around n, followed by recomputing the height in n->right,
n->left, and n.
After performing these rotations, we check whether the height of n changed by this: if not, we can finish rebalancing; otherwise we
continue with the next node up, till we reach the root.



REBALANCING A NODE IN A HEIGHT-BALANCED TREE: CASE 2.1

Since we do only$O(1)$ work on each node of the path, at most two rotations and at most three recomputations of the height, and the path has length $O(\log n)$, these rebalancing operations take only $O(\log n)$ time. But we still have to show that they do restore the height balancedness.





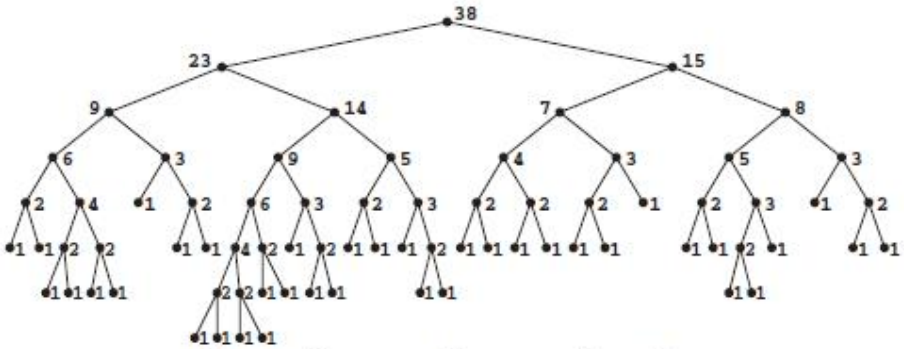
REBALANCING A NODE IN A HEIGHT-BALANCED TREE: CASE 2.2

The basic delete function needs the same modifications, with the same rebalancing code while going up the tree. There is, of course, no change at all to the find function. Because we know that the height of the stack is bounded by 1.44 log n and $n < 2100$, this is a situation where an array-based stack of fixed maximum size is a reasonable choice.

Weight-Balanced Trees

The weight of a tree is the number of its leaves, so in a weight-balanced tree, the weight of the left and right subtrees in each node should be "balanced". The top-down optimal search trees constructed in this way as balanced as possible, with the left and right weights differing by at most 1; but we cannot maintain such a strong balance condition only with $O(\log n)$ rebalancing work during insertions and deletions. Instead of bounding the difference, the correct choice is to bound the ratio of the weights. This gives an entire family of balance conditions, the α-weight-balanced trees, where for each subtree the left and right sub-subtrees have each at least a fraction of α of the total weight of the subtree (and at most a fraction of $(1 - \alpha)$). An α-weight-balanced tree has necessarily small height. Again the weight information must be corrected whenever the tree is changed and is used to keep the tree weight balanced. And the information changes only by insert and delete operations, and only in those nodes on the path from the changed leaf to the root, and there only by at most 1. So, as in the heightbalanced trees (AVL trees) follow the path up to the root and restore in each node the balance condition, using inductively that below the current node the subtrees are already balanced.
If *n is the current node and we already corrected the weight of *n, there are the following cases for the rebalancing

algorithm:
1. n->left->weight ≥ α n->weight and
n->right->weight ≥ α n->weight.
In this case, no rebalancing is necessary in this node; we go up to the next node.
2. n->right->weight < α n->weight
2.1 If n->left->left->weight > (α + ε)n->weight.
Perform a right rotation around n, followed by recomputing the weight
in n->right.
2.2 Else perform a left rotation around n->left, followed by a right
rotation around n, followed by recomputing the weight in n->left
and n->right.
3. n->left->weight < α n->weight
3.1 If n->right->right->weight > (α + ε)n->weight.
Perform a left rotation around n, followed by recomputing the weight
in n->left.
3.2 Else perform a right rotation around n->right, followed by a left
rotation around n, followed by recomputing the weight in n->left
and n->right.



0.29-Weight-Balanced Tree with Node Weights

Again the basic delete function needs the same modifications, with the same rebalancing code while going up the tree, and there is no change to the find function. Because we know that the height of the stack is bounded by (log 1 1−α)−1 log n, which is less than 2.07 log n for our interval of α, and n < 2100, again an array-based stack of fixed maximum size is a reasonable choice.

(a, b)- and B-Trees

The terms (a, b)-tree and B-tree both refer to types of balanced tree data structures commonly used in databases and file systems for efficient indexing and searching. Let's break them down.

(a, b)-Tree

A (a, b)-tree is a more generalized or flexible version of the B-tree. It’s characterized by two parameters, aa and bb, that define constraints on the number of keys in each node of the tree. This tree ensures that the height of the tree is kept balanced, which makes searching and updating operations efficient.

aa: The minimum number of keys allowed in a non-root node. This ensures that a node must have at least aa keys.

bb: The maximum number of keys allowed in a node. A node can have up to bb keys.

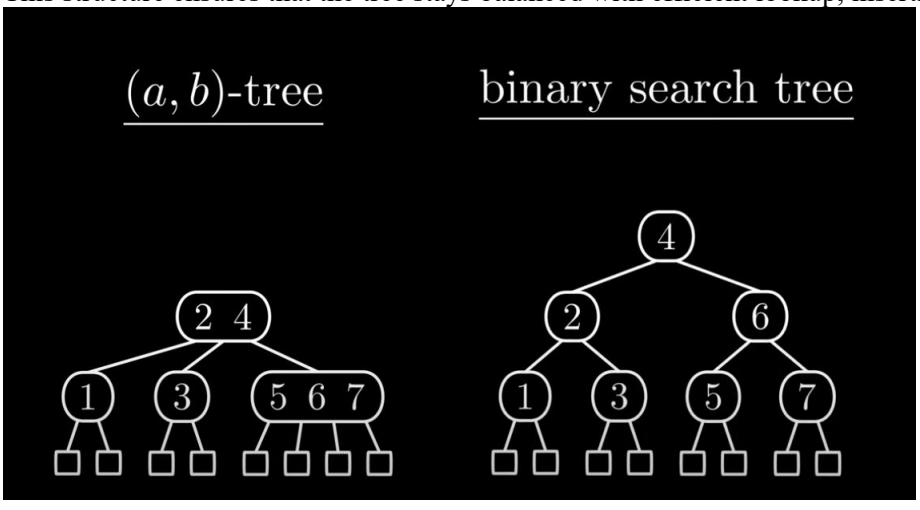
In an (a, b)-tree:

Each internal node can have between aa and bb children.

The root node may have fewer than aa children if it's not a leaf.

Every non-leaf node is required to have at least aa children, and at most bb.

This structure ensures that the tree stays balanced with efficient lookup, insertion, and deletion operations.



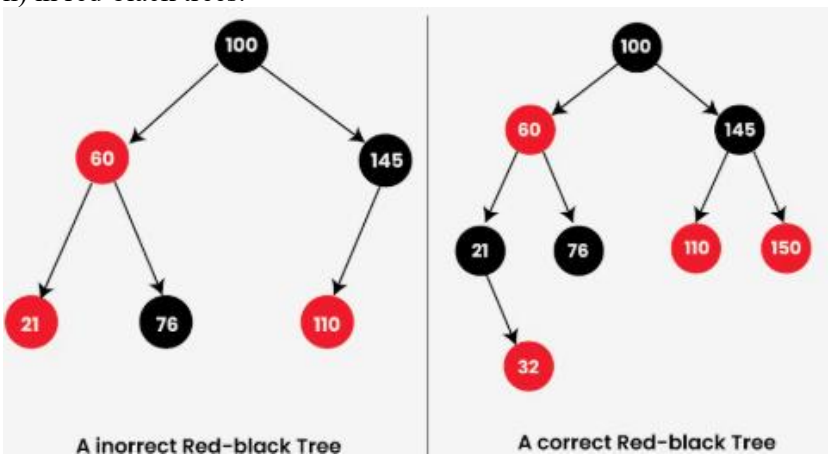
A node has the following structure:
typedef struct tr_n_t { int degree;
int height;
key_t key[B];
struct tr_n_t * next[B];
/* possibly other information */
} tree_node_t;

B-Tree
A B-tree is a self-balancing tree data structure that maintains sorted data and allows for efficient insertion, deletion, and search operations. The B-tree is used in databases and file systems for indexing. It can be considered a special case of the (a, b)-tree where the parameters aa and bb are fixed.
In a B-tree:
The order of the tree, denoted as m, is the maximum number of children a node can have. Each node in a B-tree can contain between ⌊ m/2⌋ \left\lfloor m/2 \right\rfloor and mm children, ensuring that the tree remains balanced.
Each internal node contains between ⌊ m/2⌋ \left\lfloor m/2 \right\rfloor and m−1m-1 keys.
The B-tree is always balanced, meaning the paths from the root to any leaf node are all of equal length, which guarantees logarithmic time complexity for search, insertion, and deletion.

In both cases, the primary goal is to keep the tree balanced, which ensures that operations like searching, inserting, and deleting data can be done efficiently in logarithmic time.
The time complexity of the search operation in B-Tree is O(log$_{f0}$N) because: The height of the B-Tree is O(log$_{f0}$N). At each level of the tree, searching within a node is O(1). An (a, b)-tree of height h ≥ 1 has at least 2ah−1 and at most bh leav es. An (a, b)-tree with n ≥ 2 leaves has height at most loga(n) + (1 − loga 2) ≈ 1/(log2 a) log n.

Red-Black Trees and Trees of Almost Optimal Height
Red-black trees are self-balancing binary search trees that maintain a height of approximately O(log n), ensuring efficient search, insertion, and deletion operations, while not being perfectly balanced like AVL trees. Key Properties of Red-Black Trees:
Self-Balancing: Red-black trees use a coloring scheme (red or black) to ensure that the tree remains relatively balanced during insertions and deletions.
Height Bound: The height of a red-black tree is at most 2 * $\log_2(n + 1)$, where n is the number of internal nodes. This means the tree is "almost optimal" in terms of height, ensuring logarithmic time complexity for most operations.
Red-Black Rules:
- Every node is either red or black.
- The root node is black.
- Every path from the root to a leaf contains the same number of black nodes.
- If a node is red, then both of its children must be black.

Efficiency:
The height bound of O(log n) allows for efficient search, insertion, and deletion operations, which are all typically O(log n) in red-black trees.



Trees of Almost Optimal Height
Trees that are nearly as tall as possible, given their environment and resources, maximizing their chances of reaching the canopy and reproductive success.
Optimal Height in Trees:
Trees strive for a height that allows them to access sunlight and resources effectively, leading to better growth and reproductive success.
Factors Influencing Optimal Height:
Light Availability: Trees in dense forests or under shaded conditions may have a reduced optimal height compared to

those in open areas.
Resource Competition: Competition for resources like water and nutrients can also influence the optimal height a tree can achieve.
Wood Density: Trees with lower wood density tend to have a larger diameter at a given height and grow taller than trees with higher wood density.
Near-Optimal Height:
Trees that are "almost optimal" in height are those that have reached a height that is close to the maximum height they can achieve under their specific environmental conditions.
Top-down rebalancing in the context of Red-Black Trees (RBTs) refers to the approach of rebalancing the tree starting from the root and working down to the leaves, in contrast to bottom-up rebalancing, which starts from a node and propagates changes upward.
Top-Down Rebalancing Process:
In a Red-Black Tree, when you perform insertion or deletion, the tree might become unbalanced, violating one or more of the Red-Black properties. Rebalancing ensures that these properties are restored. Top-down rebalancing works by starting the rebalancing process at the root and making adjustments as you go down the tree, as opposed to making adjustments and then propagating changes upward from the affected node.
1. **Insertion and Rebalancing** (Top-Down):
When inserting a new node into a Red-Black Tree, you follow the usual insertion process for binary search trees. After insertion, you need to check if any of the Red-Black properties are violated and rebalance the tree.
Red-Black Property Violations:
If the parent node is red (violating the red node property), then you may need to perform rotations or recoloring to fix the violations.
Top-down rebalancing typically uses rotations and recoloring to fix violations. Here's a general idea of the process:
Recoloring: If both the parent and the uncle are red, recolor both the parent and the uncle to black, and the grandparent node to red. After recoloring, move the current node up to the grandparent and continue rebalancing upwards.
Rotation: If either the parent or the uncle is black, you need to perform rotations. The exact rotation (left or right) depends on the structure of the tree. If the parent and current node are both on the same side (both left or both right), a single rotation (right or left) will suffice. If they are on opposite sides, a double rotation (first in the opposite direction, then in the desired direction) is required.
Left Rotation: A left rotation is performed when a node's right child is its successor in the BST, and we want to "push" the node left.
Right Rotation: A right rotation is performed when a node's left child is its predecessor in the BST, and we want to "push" the node right.
2. **Deletion and Rebalancing** (Top-Down):
Deletion of nodes in Red-Black Trees is more complicated, as it can cause a violation of the black height property. After a deletion, rebalancing may be needed to maintain the Red-Black Tree properties.
The main challenge after a deletion is when you remove a black node, which can potentially reduce the black height of the tree. This can lead to violations that need to be fixed.
The top-down deletion rebalancing typically involves:
Case 1: If the sibling of the node to be deleted is red, perform a rotation to push the red sibling down and move the violation up.
Case 2: If the sibling is black and its children are black, you perform a recoloring operation to maintain the black height.
Case 3: If the sibling is black and at least one of its children is red, a rotation and recoloring are performed.